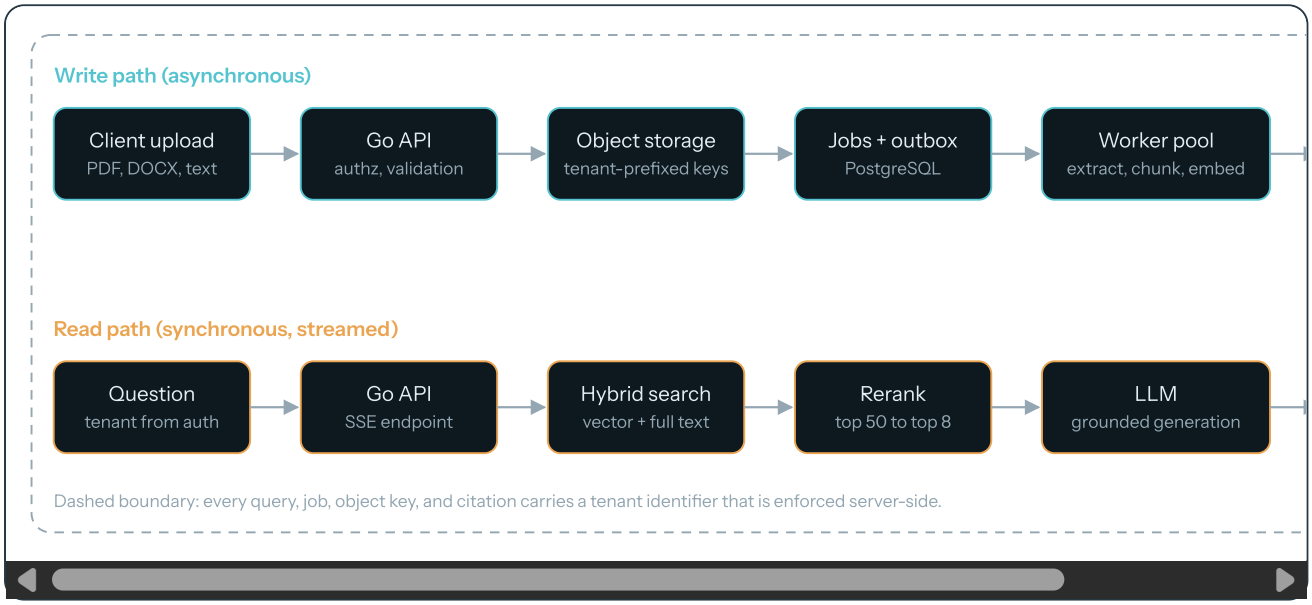
Build a production-grade multi-tenant RAG platform in Go

More than a document chatbot: an asynchronous, observable Retrieval-Augmented Generation platform that can serve many companies, teams, or customers without leaking one tenant's data into another's answers. This guide covers the architecture, how each part works, how data moves, and a milestone path to build it.



The two paths share one index. Everything left of the index is about getting trustworthy data in; everything right is about getting a grounded, attributable answer out.

1. Goals | 2. Architecture | 3. Technology choices | 4. Multi-tenancy
5. Data model | 6. Ingestion pipeline | 7. Jobs and messaging | 8. Hybrid retrieval
9. Generation and citations | 10. Observability | 11. Evaluation
12. Security and reliability | 13. Project structure | 14. Milestones
15. Testing strategy | 16. Portfolio deliverables | 17. Guardrails and red teaming

1. What you are building

A multi-tenant RAG platform lets many customers upload their own documents and ask questions answered only from that customer's data. The hard parts are not the LLM call. They are isolation, durable background processing, retrieval quality, cost control, and being able to explain what the system did on any request.

Capabilities

- Authenticated users who belong to one or more tenants, with roles.
- Document upload with validation, durable storage, and asynchronous processing.
- Hybrid retrieval (semantic vectors plus full-text) fused with Reciprocal Rank Fusion.
- Grounded answers streamed over Server-Sent Events, with citations that resolve to authorized chunks.
- Quotas, cost accounting, tracing, metrics, and failure recovery.

Non-goals at the start

- Training or fine-tuning models.
- A polished front end. A minimal demo client is enough.
- Multi-region deployment, multiple vector databases, or a full service mesh.

---

2. Architecture overview

Three Go binaries share the same internal packages. They scale and fail independently.

Binary	Responsibility
`cmd/api`	HTTP API: authentication, tenant resolution, document CRUD, upload, search and ask endpoints, SSE streaming.
`cmd/worker`	Claims jobs and runs the ingestion pipeline: extract, chunk, embed, index. Also handles deletion cleanup.
`cmd/outbox-relay`	Publishes outbox rows to a broker (optional until you introduce one).

Write path

1. The client uploads a file. The API checks the user's membership in the tenant,

enforces size limits, and validates type by signature as well as MIME.
2. The file is stored under a tenant-prefixed object key.
3. In a single database transaction, the API inserts the document row and enqueues an ingestion job.
4. A worker claims the job, extracts text, chunks it, embeds the chunks in batches, and writes chunks and vectors.
5. The document status moves to `indexed`, or to `failed` with a recorded reason.

Read path

1. The API resolves the tenant from the authenticated identity, never from an unchecked request field.
2. It embeds the question, runs vector and full-text search scoped to the tenant, and fuses the rankings.
3. An optional reranker narrows the candidates.
4. The selected chunks are numbered and placed into a grounded prompt.
5. Tokens stream to the client; citations are resolved from the retrieval records and returned.

---

3. Technology choices

Pick boring, well-understood tools first. Each choice below can be swapped later behind an interface.

Concern	Start with	Why
HTTP	`net/http` with the standard mux, or chi	Small surface, easy to test, plenty of middleware.
Database access	`pgx` (optionally with sqlc)	Native Postgres features: transactions, `set_config`, pgvector types.
Migrations	goose or golang-migrate	Versioned, reviewable schema changes.
Vectors	PostgreSQL with pgvector	One datastore keeps tenancy, transactions, and deletion simple.
Keyword search	Postgres `tsvector` + GIN	Covers exact terms, identifiers, and rare words that embeddings miss.

Files	S3-compatible storage (MinIO locally)	Same API in development and production.
Jobs	PostgreSQL-backed queue first	Transactional enqueue, no extra infrastructure. Add Kafka, RabbitMQ, or NATS only when you can justify it.
LLM and embeddings	Your own provider interfaces	Swap hosted and local models without touching retrieval.
Telemetry	OpenTelemetry, Prometheus, Grafana	Vendor-neutral traces and metrics.

4. Multi-tenancy model

Use a shared schema where every tenant-owned row has a `tenant_id`. Defend it in three layers so a single bug does not become a data leak.

1. **Application authorization.** Resolve the tenant from the authenticated identity and verify membership and role on every request.
2. **Row-level security (RLS).** The database refuses to return or write rows outside the current tenant, even if a query forgets its `WHERE` clause.
3. **Storage isolation.** Object keys are prefixed, for example `tenants/{tenant_id}/documents/{document_id}`, and access is checked before any signed URL is issued.

migrations/0002_rls.sql

Copy

```
ALTER TABLE documents ENABLE ROW LEVEL SECURITY;
ALTER TABLE documents FORCE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation ON documents
  USING      (tenant_id = current_setting('app.tenant_id', true)::uuid)
  WITH CHECK (tenant_id = current_setting('app.tenant_id', true)::uuid);

-- Repeat for chunks, jobs, usage_events, and every tenant-owned table.
```

internal/storage/tenant_tx.go

Copy

```
func (s *Store) WithTenant(ctx context.Context, tenantID uuid.UUID, fn func(pgx.T
    tx, err := s.pool.Begin(ctx)
    if err ≠ nil {
        return err
```

```
        }
        defer tx.Rollback(ctx)

        // is_local = true: the setting lives only for this transaction,
        // so a pooled connection never carries a tenant into the next request.
        if _, err := tx.Exec(ctx, "SELECT set_config('app.tenant_id', $1, true)", ten
            return err
        }
        if err := fn(tx); err != nil {
            return err
        }
        return tx.Commit(ctx)
    }
```

> **Three common RLS mistakes**
>
> Connecting as a superuser or the table owner bypasses RLS unless you use `FORCE ROW LEVEL SECURITY` and a dedicated application role. Setting the tenant outside a transaction leaks it across pooled connections. Workers and background jobs also need a tenant context; give them the same wrapper.

---

5. Data model

Start with the smallest schema that supports the pipeline. Record which embedding model produced each vector so you can re-embed later.

migrations/0001_init.sql

Copy

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE tenants (
  id         uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name       text NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE users (
  id    uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  email text NOT NULL UNIQUE
);

CREATE TABLE memberships (
  tenant_id uuid NOT NULL REFERENCES tenants(id),
  user_id   uuid NOT NULL REFERENCES users(id),
  role      text NOT NULL CHECK (role IN ('owner','admin','member')),
```

```
  PRIMARY KEY (tenant_id, user_id)
);

CREATE TABLE documents (
  id           uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id    uuid NOT NULL REFERENCES tenants(id),
  filename     text NOT NULL,
  content_type text NOT NULL,
  size_bytes   bigint NOT NULL,
  object_key   text NOT NULL,
  sha256       text NOT NULL,
  status       text NOT NULL DEFAULT 'uploaded'
               CHECK (status IN ('uploaded','queued','processing','indexed','fail
  error        text,
  created_at   timestamptz NOT NULL DEFAULT now(),
  deleted_at   timestamptz,
  UNIQUE (tenant_id, sha256)
);

CREATE TABLE chunks (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   uuid NOT NULL REFERENCES tenants(id),
  document_id uuid NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
  ordinal     int  NOT NULL,
  content     text NOT NULL,
  token_count int  NOT NULL,
  page        int,
  heading     text,
  embed_model text NOT NULL,
  embedding   vector(1536) NOT NULL,   -- match your embedding model's dimension
  tsv         tsvector GENERATED ALWAYS AS (to_tsvector('english', content)) STOF
  UNIQUE (document_id, ordinal)        -- makes retries idempotent
);

CREATE INDEX chunks_embedding_hnsw ON chunks USING hnsw (embedding vector_cosine_
CREATE INDEX chunks_tsv_gin        ON chunks USING gin (tsv);
CREATE INDEX chunks_tenant_doc     ON chunks (tenant_id, document_id);

CREATE TABLE jobs (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id       uuid NOT NULL REFERENCES tenants(id),
  kind            text NOT NULL,
  payload         jsonb NOT NULL,
  status          text NOT NULL DEFAULT 'queued'
                  CHECK (status IN ('queued','running','succeeded','failed','dea
  attempts        int  NOT NULL DEFAULT 0,
  max_attempts    int  NOT NULL DEFAULT 5,
  run_at          timestamptz NOT NULL DEFAULT now(),
  lease_expires_at timestamptz,
  locked_by       text,
```

```
  idempotency_key  text,
  last_error       text,
  UNIQUE (tenant_id, idempotency_key)
);

CREATE TABLE outbox (
  id           bigserial PRIMARY KEY,
  tenant_id    uuid NOT NULL,
  topic        text NOT NULL,
  payload      jsonb NOT NULL,
  created_at   timestamptz NOT NULL DEFAULT now(),
  published_at timestamptz
);

CREATE TABLE usage_events (
  id            bigserial PRIMARY KEY,
  tenant_id     uuid NOT NULL,
  kind          text NOT NULL,       -- embed, generate, rerank
  input_tokens  int  NOT NULL DEFAULT 0,
  output_tokens int  NOT NULL DEFAULT 0,
  created_at    timestamptz NOT NULL DEFAULT now()
);
```

> **Filtered vector search**
>
> An approximate index scans nearest neighbors first and applies the tenant filter afterward, so a small tenant can get fewer results than requested. Check whether your pgvector version offers iterative index scans, test with your smallest tenant, and consider partitioning for very large tenants.

---

6. Ingestion pipeline

Treat ingestion as a state machine. Each stage is idempotent, so a crash and retry cannot corrupt a document.

Stage	What happens	Design notes
Accept	Validate, store the object, insert document and job in one transaction.	Hash the file to detect duplicates per tenant. Never enqueue a job for a document row that failed to commit.
Extract	Convert the file to text with page and heading metadata.	Run parsers in isolation with memory and time limits. Parsers are a common attack surface.

		surface.
Chunk	Split into passages of roughly 300 to 800 tokens with 10 to 15 percent overlap.	Prefer paragraph and heading boundaries. Keep the ordinal stable so reprocessing upserts the same rows.
Embed	Send chunks to the embedding provider in batches.	Validate each returned vector's dimension. Record the model name. Respect rate limits with bounded retries.
Index	Write chunks and vectors, then mark the document `indexed`.	Do both in one transaction and check the document has not been deleted meanwhile.

Chunk size is a tuning knob, not a constant. Start with the ranges above and let your evaluation set (section 11) tell you whether to change them.

---

7. Jobs and messaging

A PostgreSQL-backed queue gives you transactional enqueue and is enough for a long time. Workers claim jobs with a lease so abandoned work is recovered automatically.

`internal/jobs/claim.sql`

Copy

```
UPDATE jobs
SET status = 'running',
    locked_by = $1,
    lease_expires_at = now() + interval '60 seconds',
    attempts = attempts + 1
WHERE id = (
  SELECT id FROM jobs
  WHERE (status = 'queued'  AND run_at ⩽ now())
     OR (status = 'running' AND lease_expires_at < now())
  ORDER BY run_at
  FOR UPDATE SKIP LOCKED
  LIMIT 1
)
RETURNING id, tenant_id, kind, payload, attempts, max_attempts;
```

While a job runs, the worker extends its lease with a heartbeat. If attempts reach `max_attempts`, mark the job `dead` and surface it for inspection rather than retrying forever. Retry delays should grow exponentially with jitter.

`cmd/worker/main.go (core loop)`

```
sem := make(chan struct{}, cfg.Concurrency) // bounded worker pool
var wg sync.WaitGroup

for ctx.Err() == nil {
    sem <- struct{}{} // reserve capacity before claiming, so a claimed job never
    job, err := queue.Claim(ctx, workerID)
    if err != nil { // includes "no jobs available"
        <-sem
        sleepWithJitter(ctx, cfg.PollInterval)
        continue
    }
    wg.Add(1)
    go func() {
        defer wg.Done()
        defer func() { <-sem }()
        w.Process(ctx, job) // heartbeat, run stages, mark succeeded or schedule
    }()
}
wg.Wait() // graceful shutdown: finish in-flight jobs
```

Copy

When you add a broker

Use the transactional outbox pattern. Write an outbox row in the same transaction as the state change, and let `outbox-relay` publish it. Delivery is at-least-once, so every consumer must be idempotent: use idempotency keys and unique constraints so a duplicate event does nothing.

---

8. Hybrid retrieval

Vector search finds passages that mean the same thing as the question. Full-text search finds exact names, error codes, and rare terms. Run both and fuse the rankings with Reciprocal Rank Fusion: each result scores the sum of `1 / (k + rank)` across lists, with `k` commonly 60. It needs no score normalization, which is why it works well across two very different scoring systems.

`internal/retrieval/hybrid.sql`

```
WITH vec AS (
  SELECT id, row_number() OVER (ORDER BY embedding <=> $2) AS rnk
  FROM chunks
  WHERE tenant_id = $1
  ORDER BY embedding <=> $2
  LIMIT 50
),
```

Copy

```
  fts AS (
    SELECT id, row_number() OVER (ORDER BY ts_rank_cd(tsv, q) DESC) AS rnk
    FROM chunks, websearch_to_tsquery('english', $3) AS q
    WHERE tenant_id = $1 AND tsv @@ q
    ORDER BY ts_rank_cd(tsv, q) DESC
    LIMIT 50
  )
  SELECT c.id, c.document_id, c.content, c.page, c.heading,
         COALESCE(1.0 / (60 + vec.rnk), 0) + COALESCE(1.0 / (60 + fts.rnk), 0) AS s
  FROM chunks c
  LEFT JOIN vec ON vec.id = c.id
  LEFT JOIN fts ON fts.id = c.id
  WHERE c.tenant_id = $1
    AND (vec.id IS NOT NULL OR fts.id IS NOT NULL)
  ORDER BY score DESC
  LIMIT $4;
```

Join to `documents` and exclude soft-deleted or `deleting` documents so removed content can never be retrieved. Then optionally rerank the top candidates with a cross-encoder or an LLM, keeping the best handful for the prompt. Measure whether reranking earns its latency and cost before keeping it.

---

9. Generation, streaming, and citations

Grounded prompt

Number each retrieved chunk and tell the model to answer only from them, cite by number, and say so when the context is insufficient. Retrieved text goes in as clearly delimited data. It is untrusted, however well it is labelled.

Streaming over SSE

`internal/conversations/handler.go`

Copy

```
  func (h *Handler) Ask(w http.ResponseWriter, r *http.Request) {
      fl, ok := w.(http.Flusher)
      if !ok {
          http.Error(w, "streaming unsupported", http.StatusInternalServerError)
          return
      }
      ctx := r.Context() // cancelled automatically when the client disconnects
      tenant := auth.TenantFrom(ctx)

      hits, err := h.retriever.Search(ctx, tenant, question)
```

```
        if err ≠ nil {
            http.Error(w, "retrieval failed", http.StatusBadGateway)
            return
        }

        w.Header().Set("Content-Type", "text/event-stream")
        w.Header().Set("Cache-Control", "no-cache")

        for ev := range h.gen.Stream(ctx, buildRequest(question, hits)) {
            if ev.Err ≠ nil {
                writeSSE(w, "error", []byte(`{"message":"generation failed"}`))
                fl.Flush()
                return
            }
            writeSSE(w, ev.Kind, ev.Data) // "token", "citation", or "done"
            fl.Flush()
        }
    }

    func writeSSE(w io.Writer, event string, data []byte) {
        fmt.Fprintf(w, "event: %s\ndata: %s\n\n", event, data)
    }
```

Pass the request context all the way into the provider call so a disconnected client stops spending tokens. Record final token usage in `usage_events` when the stream ends.

Citations you can trust

The model only emits markers like `[2]`. Your code maps each marker back to the chunk records you retrieved for this request, drops any marker that does not exist, and re-checks access when a citation is later resolved to a document or page. Never let the model invent a source identifier.

---

10. Observability

The goal: explain the latency and failure of any request from its telemetry alone. Propagate trace context from the API through the job payload into the worker.

Signal	What to capture
Query trace	Spans for embed question, vector search, full-text search, fusion, rerank, and LLM first-token and total time.
Ingestion	Spans for claim, extract, chunk, embed batches, and index write, linked to

Ingestion trace	Spans for claim, extract, chunk, embed batches, and index write, linked to the upload request.
Metrics	Request latency histograms, job queue depth and age, job attempts and failures, provider errors and rate limits, tokens and estimated cost per tenant.
Logs	Structured, with request, tenant, document, and job identifiers. Never log document text or secrets.

Keep metric label cardinality in check: tenant identifiers are fine for a few hundred tenants, but document or user identifiers belong in traces and logs, not metric labels.

---

11. Evaluation

Without a test set, "better retrieval" is a feeling. Build a small golden dataset early and run it on every change.

`tests/evaluation/golden.jsonl`

Copy

```
{"tenant":"acme","question":"What is the refund window?","relevant_chunks":["refu
{"tenant":"acme","question":"Who won the 2018 World Cup?","relevant_chunks":[],"r
```

- **Retrieval:** recall@k, MRR, and nDCG against the labelled chunks.
- **Answers:** faithfulness to the retrieved context and correctness against the reference. LLM judges help, but spot-check them by hand.
- **Citations:** every marker maps to a real, authorized chunk that supports the sentence.
- **Refusals:** out-of-scope questions are declined rather than answered from model memory.

---

12. Production security and reliability

A system can produce accurate answers and still be unsafe. These are the areas you should treat as production requirements.

Risk	Mitigation
Cross-tenant data	Tenant-aware queries, authorization checks, RLS, isolation tests

leakage	
Malicious uploaded files	File-size limits, MIME and signature validation, parser isolation, malware scanning where appropriate
Prompt injection in documents	Treat retrieved content as untrusted; restrict tools and privileged actions
Unauthorized citations	Generate citations from authorized retrieval records and verify access when resolving them
Duplicate jobs	Idempotency keys, unique constraints, transactional state changes
Provider outages	Timeouts, bounded retries, exponential backoff, circuit breaking where useful
Excessive LLM spending	Per-tenant quotas, concurrency limits, token budgets, cost monitoring
Memory exhaustion	Bounded worker pools, streaming parsers where possible, document and chunk limits
Stuck jobs	Leases, heartbeats, recovery of abandoned work, retry limits
Accidental deletion	Authorization, soft deletion where appropriate, retention and backup policies

Section 17 expands this table into a layered guardrail design and a red-team playbook. Two details deserve special attention first.

> **Prompt injection**
>
> A document may contain text instructing the model to reveal secrets or ignore its system prompt. Prompt instructions help, but they are not a complete security boundary. Do not expose privileged tools merely because the model asks to use them. Enforce authorization in application code and require explicit validation before consequential actions.

> **Deletion**
>
> Deleting a document means more than removing its metadata. You must also clean up its chunks, vectors, cached content, associated objects, and any derived artifacts according to your retention policy. Account for in-flight jobs so that a worker cannot reinsert deleted content after cleanup finishes.

> that a worker cannot reinsert deleted content after cleanup finishes.

---

13. Recommended Go project structure

Organize the code around responsibilities rather than putting the entire application in `main.go`.

Copy

```
contexthub/
├── cmd/
│   ├── api/
│   │   └── main.go
│   ├── worker/
│   │   └── main.go
│   └── outbox-relay/
│       └── main.go
├── internal/
│   ├── auth/
│   ├── tenants/
│   ├── documents/
│   ├── ingestion/
│   │   ├── extract/
│   │   ├── chunk/
│   │   └── embed/
│   ├── jobs/
│   ├── messaging/
│   ├── retrieval/
│   ├── reranking/
│   ├── llm/
│   ├── conversations/
│   ├── observability/
│   └── storage/
├── migrations/
├── api/
│   └── openapi.yaml
├── deployments/
│   └── docker-compose.yml
├── tests/
│   ├── integration/
│   ├── isolation/
│   └── evaluation/
├── scripts/
├── docs/
│   └── architecture.md
├── go.mod
├── go.sum
├── Makefile
└── README.md
```

This is a suggested layout, not a requirement to create every directory on day one.

Keep your interfaces small. For example, the RAG application should depend on an embedding interface rather than a particular provider's SDK.

`internal/llm/llm.go`

Copy

```
package llm

import "context"

type EmbeddingProvider interface {
    Embed(
        ctx context.Context,
        texts []string,
    ) ([][]float32, error)
}

type GenerationRequest struct {
    SystemPrompt string
    Question     string
    Context      []string
}

type TokenUsage struct {
    InputTokens  int
    OutputTokens int
}

type Generator interface {
    Generate(
        ctx context.Context,
        req GenerationRequest,
    ) (←chan string, ←chan error)
}
```

This is an illustrative interface. In a real streaming API, prefer a single stream abstraction that represents both token events and terminal errors, with explicit cancellation and final usage reporting. Also validate that every returned embedding has the expected dimension.

The benefit of provider abstractions is that you can switch between a hosted model and a local model without rewriting your retrieval pipeline.

---

14. How to implement it without getting overwhelmed

Build the system in increments. Do not start by launching Kafka, RabbitMQ, multiple vector databases, and a full monitoring stack on the same day. Tick milestones off as you finish them; progress is saved in your browser.

0 of 5 milestones complete

1 Working Go API

Implement authentication, tenant membership, document metadata, PostgreSQL migrations, CRUD endpoints, and tests.

Exit criterion: tenant-scoped API tests pass.

☐ Complete

2 Asynchronous ingestion

Add object storage, durable jobs, a worker pool, extraction, chunking, embeddings, and indexing. Start with PostgreSQL-backed jobs or one broker.

Exit criterion: restarting a worker does not corrupt document state.

☐ Complete

3 RAG works end to end

Add vector retrieval, full-text search, rank fusion, grounded generation, streaming, and citations.

Exit criterion: a test dataset demonstrates measurable retrieval and answer quality.

☐ Complete

4 Observable production behavior

Add traces, metrics, dashboards, cost accounting, timeouts, quotas, and load tests.

Exit criterion: you can explain the latency and failure of a request using its telemetry.

☐ Complete

5 Release-quality capstone

Complete security tests, failure recovery, deployment documentation,

reproducible benchmarks, and a working demo.

Exit criterion: another developer can run the project using your documented setup.

- [ ] Complete

---

15. Testing strategy

You should have four distinct types of tests.

Unit tests

- Chunk boundaries and metadata extraction.
- Retry and backoff decisions.
- RRF ranking and citation mapping.
- Cost calculations and usage aggregation.

Integration tests

- PostgreSQL queries and migrations.
- File storage and document processing.
- Message publishing and consumption.
- Embedding and generation adapters.
- SSE behavior and client cancellation.

Security tests

- Cross-tenant document access.
- Cross-tenant retrieval and citations.
- Unauthorized job inspection and deletion.
- Forged tenant identifiers.
- Prompt injection and tool authorization.

Failure and load tests

- Kill a worker while it processes a job.
- Disconnect the broker or database temporarily.
- Simulate provider rate limits and timeouts.

- Send duplicate events.
- Upload malformed and oversized files.
- Generate concurrent requests and measure p95 latency, error rate, and resource usage.

For Go, run `go test ./...` and `go test -race ./...` regularly. The race detector can catch certain memory-access races, but it does not prove that your distributed job processing is correct.

---

16. What should your final portfolio deliver?

By the end, aim to publish a repository containing:

- A functioning Go API and worker service.
- PostgreSQL migrations and a documented tenant model.
- Asynchronous document ingestion with retry and recovery behavior.
- Hybrid RAG with citations and streamed answers.
- A golden dataset with retrieval and answer evaluations.
- OpenTelemetry instrumentation and Prometheus dashboards.
- Docker Compose configuration and reproducible setup instructions.
- Automated tests for isolation, failures, and concurrency.
- A red-team suite with documented attack results and the guardrails each finding led to (section 17).
- A load-test report with actual measurements.
- An architecture diagram, demo, and technical write-up.

The most compelling portfolio story is not simply that you used Go, Kafka, pgvector, and an LLM. It is that you made the system reliable, measured its behavior, tested its security boundaries, and can explain the engineering decisions behind it.

> **Your immediate next step**
>
> Start with Milestone 1: Go API + PostgreSQL + multi-tenancy. Get that correct before introducing the queue.
>
> Once the tenant model and document API are stable, asynchronous ingestion and RAG become much easier to develop and test independently. This also gives you a useful working project at every stage instead of waiting until week 16

> gives you a useful working project at every stage instead of waiting until week 16 to have something demonstrable.

---

17. Guardrails and red teaming

Guardrails are the controls that keep the system inside safe behavior. Red teaming is the practice of attacking your own system to find where those controls fail. You need both: guardrails without attacks are untested assumptions, and attacks without guardrails are just a list of problems.

> **The core rule**
> The model is not a security boundary. Anything a model is merely asked to do ("never reveal other tenants' data") can be talked out of doing. Put hard guarantees in deterministic code and the database. Use model-based checks as an extra, probabilistic layer, and measure them as such.

Guardrail layers

Apply defense in depth. Each layer assumes the one before it can fail.

Layer	Guardrails	Enforced by
Input	Per-user and per-tenant rate limits, maximum question length, maximum upload size, allowed file types, abuse detection on repeated probing.	API middleware
Retrieval	Tenant filter and RLS, document-level permissions where needed, soft-deleted documents excluded, a cap on chunks per document so one file cannot dominate results.	SQL and application code
Prompt assembly	Retrieved text delimited as untrusted data, no secrets or keys in prompts, context size budget.	Application code
Tools and actions	Least privilege, an allowlist of tools, schema validation of arguments, tenant-bound credentials, human confirmation for consequential actions. The model proposes; your code decides.	Application code
Output	Citation verification, groundedness and refusal checks, redaction of secret and personal-data patterns, no automatic loading of model-written images or links,	Output guard stage

	HTML escaping in the client.	
Budget	Token limits per request, spend quotas per tenant, concurrency caps, circuit breakers on providers.	Middleware and quota service

Model the checks as a small pipeline of stages. Fail closed: if a guard errors or times out, block the response rather than letting it through.

internal/guard/guard.go

Copy

```
package guard

type Verdict int

const (
    Allow Verdict = iota
    Redact
    Block
)

type Guard interface {
    Name() string
    // Returns the verdict and the (possibly cleaned) text.
    Check(ctx context.Context, text string) (Verdict, string, error)
}

// Run executes stages in order. The first Block wins; Redact output feeds the ne
func Run(ctx context.Context, text string, stages ...Guard) (Verdict, string, err
    final := Allow
    for _, g := range stages {
        v, out, err := g.Check(ctx, text)
        if err ≠ nil {
            return Block, "", fmt.Errorf("guard %s: %w", g.Name(), err) // fail c
        }
        switch v {
        case Block:
            return Block, "", nil
        case Redact:
            final, text = Redact, out
        }
    }
    return final, text, nil
}
```

Citation verification is the most valuable deterministic output guard. The model may only point at chunks you retrieved for this request.

internal/conversations/citations.go

internal/conversations/citations.go

Copy

```
var marker = regexp.MustCompile(`\[(\d+)\]`)

// Keep only citations that point at chunks retrieved for THIS request and tenant
func ResolveCitations(answer string, hits []Hit) []Citation {
    seen := map[int]bool{}
    var out []Citation
    for _, m := range marker.FindAllStringSubmatch(answer, -1) {
        n, _ := strconv.Atoi(m[1])
        if n < 1 || n > len(hits) || seen[n] {
            continue // invented or duplicate marker
        }
        seen[n] = true
        out = append(out, Citation{N: n, ChunkID: hits[n-1].ID, DocumentID: hits[
    }
    return out
}
```

Threat model

Before attacking, write down who you defend against and what they want.

Attacker	Goal	Entry point
Malicious user inside a tenant	Read another tenant's data, escalate role, extract the system prompt	Questions, IDs in URLs, headers, tokens
Malicious uploader	Hijack answers for other users in the same tenant, crash workers	Uploaded documents
Curious or hostile other tenant	Cross-tenant leakage through shared infrastructure	Caches, indexes, job queues, object storage
External abuser	Run up your LLM bill or take the service down	Public endpoints, stolen or free accounts

Useful references for structuring this work include the OWASP Top 10 for LLM Applications and MITRE ATLAS. Check their current versions rather than relying on remembered category numbers.

Red-team playbook

Run these against your own system, in an environment you own, with test tenants. Each row is a category of test with a clear pass condition.

Category	What the red team tries	Pass condition
Direct prompt injection	Questions that tell the model to ignore its rules, reveal its instructions, or change roles.	Rules hold; no system prompt or secret appears in the answer.
Indirect prompt injection	Documents containing instructions aimed at the model, including hidden text (white on white, tiny fonts, metadata fields).	The instructions are not followed and no privileged action is triggered.
Cross-tenant access	Forged tenant IDs in headers, bodies, and token claims; guessing document, job, and chunk IDs; resolving another tenant's citation; reusing signed URLs.	Always denied. Zero tolerance; tested deterministically.
Cache and shared-state leakage	Asking questions designed to hit cached answers or embeddings belonging to another tenant.	All cache keys include the tenant; no cross-tenant hits.
Data exfiltration	Getting the model to emit links or images whose URLs carry retrieved content to an outside server.	Model output cannot trigger outbound requests with private data.
Retrieval poisoning	Uploading keyword-stuffed or look-alike documents so they outrank genuine sources.	Per-document caps and provenance limit the damage; upload rights are enforced.
Resource abuse (denial of wallet)	Huge or malformed files, decompression bombs, very long prompts, floods of queries.	Limits trigger early; quotas stop spend; workers stay within memory bounds.
Policy bypass and off-scope answers	Asking for content outside the product's purpose or answers not in the documents.	The system refuses or says the context is insufficient.
Auth and role abuse	Tampered tokens, expired tokens, member calling admin endpoints.	Rejected on every endpoint, including deletion and job inspection.
Deletion races	Deleting a document while it is being processed or retrieved.	Deleted content never reappears in search or citations.

Running it as an engineering practice

- **Canary strings.** Plant a unique, meaningless token in each test tenant's documents. If tenant A's answers ever contain tenant B's canary, you have a leak.